# 4 Score Functions and Score Matching

In the last section, we showed how to train a flow model with flow matching. In this section, we discuss diffusion models and demonstrate how to train them using **score matching**.

## 4.1 Conditional and Marginal Score Functions

So far, the central object of interest for our investigation was a vector field $u_t(x)$. Diffusion models [45, 44] take a different perspective focused on **score functions**. Therefore, in this section, we will rephrase what we have learned here in the language of score functions - providing a novel perspective. Let $q(x)$ be an arbitrary probability distribution. Then the **score function** of $q$ is defined as $\nabla \log q(x)$, i.e. as the gradient of the log-likelihood of $q$ with respect to $x$. The score has an intuitive meaning: $\nabla \log q(x)$ is the direction of steepest ascent with respect to log-likelihood. This is illustrated in Figure 8.

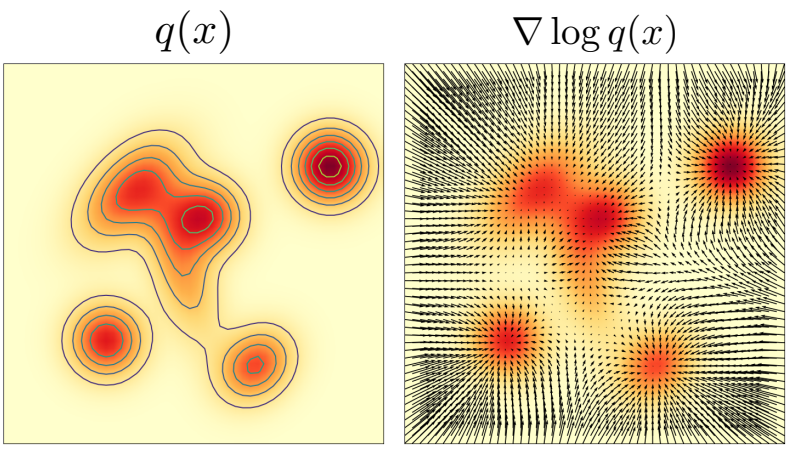



Figure 8: Illustration of score function $\nabla \log q(x)$ plotted as black rows (right) of a general probability distribution $q(x)$ (left).

Let us return to the setting of conditional probability paths $p_t(x|z)$ and marginal probability paths $p_t(x)$ as in Section 3. Then we can equivalently define the **conditional score function** as $\nabla \log p_t(x|z)$ and the **marginal score function** as $\nabla \log p_t(x)$. Similar to Equation (18), the marginal score can be expressed via the conditional score function $\nabla \log p_t(x|z)$ via

$$\nabla \log p_t(x) = \int \nabla \log p_t(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z. \tag{38}$$

Hence, **the relation between the conditional and marginal score is analogous to the relation between the conditional and marginal vector field**. Note that we can prove Equation (38) via

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)} = \frac{\nabla \int p_t(x|z) p_{\text{data}}(z) \mathrm{d}z}{p_t(x)} = \frac{\int \nabla p_t(x|z) p_{\text{data}}(z) \mathrm{d}z}{p_t(x)} = \int \nabla \log p_t(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z, \tag{39}$$

where we have used the rule $\partial_y \log y = 1/y$ combined with the chain rule twice.

**Example 15** (Score Function for Gaussian Probability Paths.)
For the Gaussian path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$, we can use the form of the Gaussian probability density (see Equation (97)) to get

$$\nabla \log p_t(x|z) = \nabla \log \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d) = -\frac{x - \alpha_t z}{\beta_t^2}. \tag{40}$$

Note that the score function for a Gaussian probability path is a linear function of $x$ and z. The same is true for the conditional vector field $u_t(x|z)$ (see Equation (20)). It is thus possible to convert between the two, as the next proposition illustrates.

**Proposition 1** (Conversion Formula for Gaussian Probability Paths)
For the Gaussian probability path $p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional (resp. marginal) vector field and the conditional (resp. marginal) score are related by the following identities

$$u_t^{\text{target}}(x|z) = a_t \nabla \log p_t(x|z) + b_t x, \quad a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right), \quad b_t = \frac{\dot{\alpha}_t}{\alpha_t} \tag{41}$$

$$u_t^{\text{target}}(x) = a_t \nabla \log p_t(x) + b_t x. \tag{42}$$

In particular, we note that the conditional (resp. marginal) vector field can be recovered from the conditional (resp. marginal) score, and vice versa.

*Proof.* For the conditional vector field and conditional score, we can derive:

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x \overset{(i)}{=} \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right)\left(\frac{\alpha_t z - x}{\beta_t^2}\right) + \frac{\dot{\alpha}_t}{\alpha_t} x = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right) \nabla \log p_t(x|z) + \frac{\dot{\alpha}_t}{\alpha_t} x$$

where in $(i)$ we just did some algebra. By taking integrals, the same identity holds for the marginal flow vector field and the marginal score function:

$$\begin{aligned} u^{\text{target}}(x) = \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z &= \int \left[a_t \nabla \log p_t(x|z) + b_t x\right] \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z \\ &\overset{(i)}{=} a_t \nabla \log p_t(x) + b_t x \end{aligned}$$

where in $(i)$ we used Equation (38) and the fact that posterior density integrates to 1. □

Proposition 1 is striking because it says that once we've learned $u_t^{\text{target}}$ we've also learned the score function $\nabla \log p_t(x)$, and vice versa. Therefore, many diffusion models learn the score function $\nabla \log p_t(x)$ instead via a neural network. We will discuss this in Section 4.3.

**Remark 16** (Reparameterization of the Score)
The reparameterization formula for Gaussian probability paths in Equation (41) is possible because both sides (conditional vector field and conditional score) are *linear* functions of $x$ and $z$. Once we marginalize (marginal vector field and marginal score), both sides are just a linear reparameterization of the posterior mean $\mathbb{E}_{z|x}[z]$. It follows that any quantity that allows to recover $\mathbb{E}_{z|x}[z]$ can in turn be used to recover the unconditional vector field and score. Further, doing so might even be preferable from a numerical/training stability standpoint. One common choice is the posterior mean itself, often referred to as the *denoiser*. Formally, we define the **conditional and marginal denoiser** as

$$D_t(x|z) = z, \quad D_t(x) = \int z \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z \overset{(i)}{=} \frac{1}{\dot{\alpha}_t \beta_t - \alpha_t \dot{\beta}_t} (\beta_t u_t^{\text{target}}(x_t) - \dot{\beta}_t x_t). \tag{43}$$

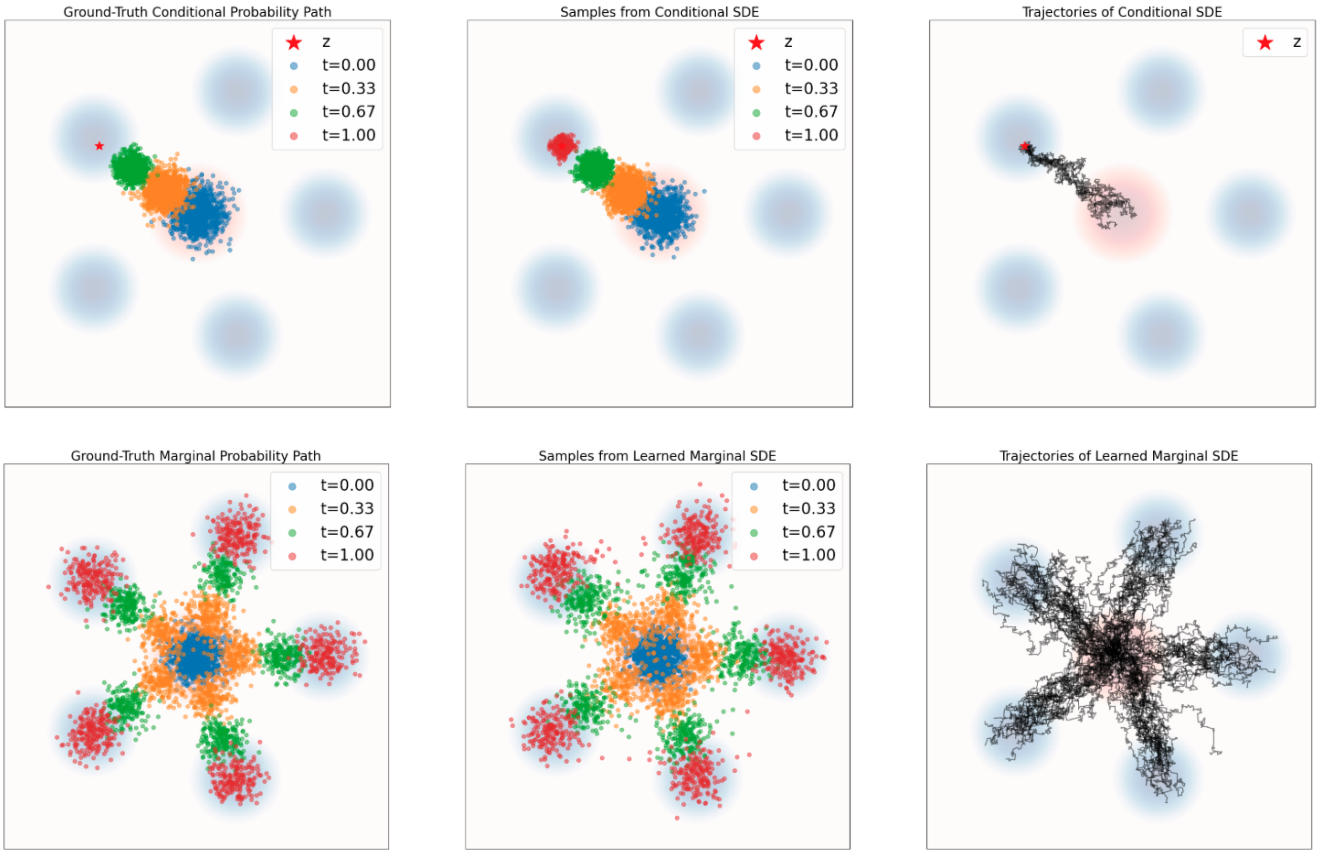



Figure 9: Illustration of Theorem 17. Simulating a probability path with SDEs. This repeats the plots from Figure 6 with SDE sampling using Equation (44). Data distribution $p_{\text{data}}$ in blue background. Gaussian $p_{\text{init}}$ in red background. Top row: Conditional path. Bottom row: Marginal probability path. As one can see, the SDE transports samples from $p_{\text{init}}$ into samples from $\delta_z$ (for the conditional path) and to $p_{\text{data}}$ (for the marginal path).

Here, $(i)$ follows from an equivalent derivation as in Proposition 1. The denoiser has a very intuitive interpretation: it is the expected value of clean data $z$ given noisy data $x$.[a] People often call such models **denoising diffusion models** as learning $D_t$ and learning $u_t^{\text{target}}$ are theoretically equivalent.

[a] Food for thought: will the denoiser always output a "clean" data point? Why or why not, and what might this depend on?

## 4.2 Sampling with SDEs

So far, we have demonstrated how one can construct a trajectory $X_t$ of an ODE that follows a desired probability path $p_t$ via a marginal vector field $u_t^{\text{target}}$. But this approach is constrained to flow models. What about diffusion models? Using score functions, let us now extend this result to SDEs.

**Theorem 17** (SDE Extension Trick)
Define the conditional and marginal vector fields $u_t^{\text{target}}(x|z)$ and $u_t^{\text{target}}(x)$ as before. Then, for any diffusion

coefficient $\sigma_t \geq 0$, we may construct an SDE by adding stochastic dynamics to the dynamics of the original ODE as follows:

$$X_0 \sim p_{\text{init}}, \qquad \mathrm{d}X_t = u_t^{\text{target}}(X_t)\mathrm{d}t + \frac{\sigma_t^2}{2}\nabla \log p_t(X_t)\mathrm{d}t + \sigma_t \mathrm{d}W_t \tag{44}$$

$$= \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2}\nabla \log p_t(X_t)\right]\mathrm{d}t + \sigma_t \mathrm{d}W_t$$

$$\Rightarrow \quad X_t \sim p_t \quad (0 \leq t \leq 1). \tag{45}$$

In particular, $X_1 \sim p_{\text{data}}$ for this SDE. We note that the stochastic dynamics are closely related to the **Langevin dynamics**, and can be thought of as injecting noise while preserving the marginal distribution $p_t$. We discuss Langevin dynamics briefly in Remark 20.

We illustrate the dynamics described in Theorem 17 in Figure 9. As one can see, the trajectories are now zig-zagged, illustrating the stochastic nature of the SDE's evolution. As Theorem 17 establishes however, the marginals $p_t$ stay the same. Note that the above result is striking in that we can choose *any* diffusion coefficient $\sigma_t \geq 0$ even after having trained the networks. In theory, Theorem 17 holds for any choice of $\sigma_t$. However, *in practice*, we suffer from both *training error* (the neural network does not perfectly approximate the marginal vector field and score) and simulation error (e.g. for $\sigma_t \gg 0$, we would need to take prohibitively small step sizes in Algorithm 2). In practice, for a fixed trained model, there is then an optimal $\sigma_t \geq 0$ which can be empirically determined [23, 1, 28].[2]

For Gaussian probability paths, we get the score function for free by having learned the marginal vector field.

**Example 18** (Gaussian SDE Extension Trick)
By Proposition 1, for Gaussian probability paths, we can express the SDE from Theorem 17 purely using score functions:

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = \left[\left(a_t + \frac{\sigma_t^2}{2}\right)\nabla \log p_t(X_t) + b_t X_t\right]\mathrm{d}t + \sigma_t \mathrm{d}W_t \tag{46}$$

$$\Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1) \tag{47}$$

where $a_t, b_t$ are defined as in Proposition 1.

In the remainder of this section, we will prove Theorem 17 via the **Fokker-Planck equation**, which extends the continuity equation from ODEs to SDEs. To do so, let us first define the **Laplacian** operator $\Delta$ via

$$\Delta w_t(x) = \sum_{i=1}^{d} \frac{\partial^2}{\partial x_i^2} w_t(x) = \operatorname{div}(\nabla w_t)(x), \tag{48}$$

for scalar field $w_t : \mathbb{R}^d \to \mathbb{R}$.

[2] Again, we stress that the existence of a "best $\sigma_t$" is an artifact of imperfectly trained models and finite compute budgets rather than a theoretical statement about the dynamics in their continuous limit.

**Theorem 19** (Fokker-Planck Equation)
Let $p_t$ be a probability path and let us consider the SDE

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = u_t(X_t)\mathrm{d}t + \sigma_t \mathrm{d}W_t.$$

Then $X_t$ has distribution $p_t$ for all $0 \leq t \leq 1$ if and only if the **Fokker-Planck equation** holds:

$$\partial_t p_t(x) = -\mathrm{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \quad \text{for all } x \in \mathbb{R}^d, 0 \leq t \leq 1, \tag{49}$$

A self-contained proof of the Fokker-Planck equation can be found in Section B. Note that Theorem 11 is recovered from the Fokker-Planck equation when $\sigma_t = 0$. The additional Laplacian term $\Delta p_t$ might be hard to rationalize at first. Those familiar with physics will note that the same term also appears in the heat equation (which is in fact a special case of the Fokker-Planck equation). Heat diffuses through a medium. We also add a diffusion process (not a physical but a mathematical one) and hence we add this additional Laplacian term. Let us now use the Fokker-Planck equation to help us prove Theorem 17.

*Proof of Theorem 17.* By Theorem 19, we need to show that that the SDE defined in Equation (44) satisfies the Fokker-Planck equation for $p_t$. We can do this by direction calculation:

$$\begin{aligned}
\partial_t p_t(x) &\overset{(i)}{=} -\mathrm{div}(p_t u_t^{\text{target}})(x) \\
&\overset{(ii)}{=} -\mathrm{div}(p_t u_t^{\text{target}})(x) - \frac{\sigma_t^2}{2}\Delta p_t(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(iii)}{=} -\mathrm{div}(p_t u_t^{\text{target}})(x) - \mathrm{div}(\frac{\sigma_t^2}{2}\nabla p_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(iv)}{=} -\mathrm{div}(p_t u_t^{\text{target}})(x) - \mathrm{div}(p_t \left[\frac{\sigma_t^2}{2}\nabla \log p_t\right])(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(v)}{=} -\mathrm{div}\left(p_t \left[u_t^{\text{target}} + \frac{\sigma_t^2}{2}\nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x),
\end{aligned}$$

where in $(i)$ we used Theorem 11, in $(ii)$ we added and subtracted the same term, in $(iii)$ we used the definition of the Laplacian (Equation (48)), in $(iv)$ we used that $\nabla \log p_t = \frac{\nabla p_t}{p_t}$, and in $(v)$ we used the linearity of the divergence operator. The above derivation shows that the SDE defined in Equation (44) satisfies the Fokker-Planck equation for $p_t$. By Theorem 19, this implies $X_t \sim p_t$ for $0 \leq t \leq 1$, as desired. □

**Remark 20** (Optional: Langevin Dynamics)
The above construction has a famous special case when the probability path is constant, i.e. $p_t = p$ for a fixed distribution $p$. In this case, we set $u_t^{\text{target}} = 0$ and obtain the SDE

$$\mathrm{d}X_t = \frac{\sigma_t^2}{2}\nabla \log p(X_t)\mathrm{d}t + \sigma_t dW_t, \tag{50}$$

which is commonly known as **Langevin dynamics**. The fact that $p_t$ is constant implies that $\partial_t p_t(x) = 0$. It follows immediately from Theorem 17 that these dynamics satisfy the Fokker-Planck equation for the static path

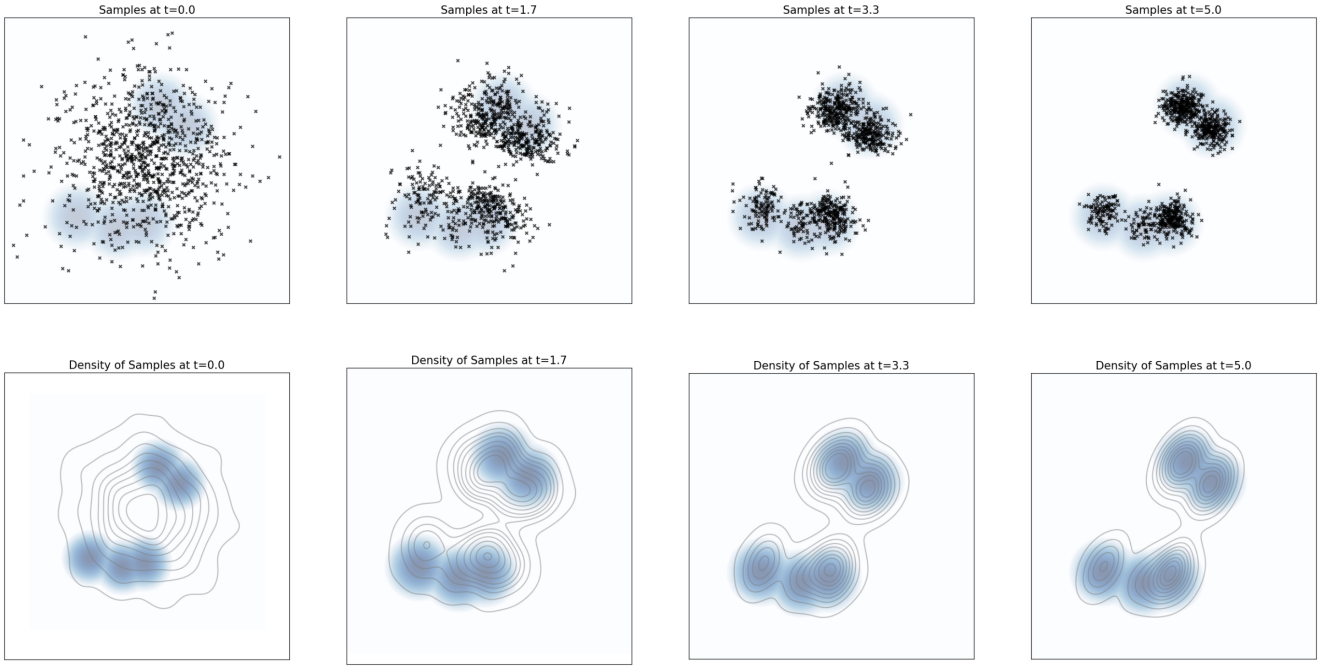



Figure 10: Top row: Particles evolving under the Langevin dynamics given by Equation (50), with $p(x)$ taken to be a Gaussian mixture with 5 modes. Bottom row: A kernel density estimate of the same samples shown in the top row. As one can see, the distribution of samples converges to the equilibrium distribution $p$ (blue background colour).

$p_t = p$ in Theorem 17. Therefore, we may conclude that $p$ is a stationary distribution of Langevin dynamics:

$$X_0 \sim p \quad \Rightarrow \quad X_t \sim p \quad (t \geq 0).$$

As with many Markov processes, these dynamics converge to the stationary distribution $p$ under rather general conditions. That is, if we instead we take $X_0 \sim p' \neq p$, so that $X_t \sim p'_t$, then under mild conditions $p_t \to p$. This fact makes Langevin dynamics extremely useful, and it accordingly serves as the basis for e.g., **molecular dynamics** simulations, and many other Markov chain Monte Carlo (MCMC) methods across Bayesian statistics and the natural sciences. In particular, the Ornstein-Uhlenbeck processes are recovered as the special case of the Langevin dynamics when $p$ is a Gaussian, and serve as the basis for initial formulations of diffusion models.

**Remark 21** (Optional: GLASS Flows, Stochastic evolution with ODEs)
The remarkable property of SDE sampling (compared to ODEs) is that the evolution becomes stochastic, i.e. the initial point $X_0$ does not fully determine $X_t$ for $t > 0$. Perhaps surprisingly, it is also possible to get the same stochastic transitions purely via ODEs via a simple sampling trick called *GLASS Flows* [20]. This allows to exploit the stochastic nature of SDEs (e.g. via search algorithms) while keeping the efficiency of ODEs.

### 4.3 Score Matching

It remains to show how we can learn the marginal score function $\nabla \log p_t(x)$. Of course, for Gaussian probability paths, we can simply transform $u_t^{\text{target}}(x)$ by Proposition 1. However, what about in general? It turns out that we can also learn marginal score functions directly. To approximate the marginal score $\nabla \log p_t$, we use a neural network that we call **score network** $s_t^\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$. In the same way as before, we can design a **score matching** loss and a **denoising score matching** loss:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{t\sim \text{Unif},\, z\sim p_{\text{data}},\, x\sim p_t(\cdot|z)} \left[ \left\| s_t^\theta(x) - \nabla \log p_t(x) \right\|^2 \right] \qquad \blacktriangleright \text{ score matching loss}$$

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{t\sim \text{Unif},\, z\sim p_{\text{data}},\, x\sim p_t(\cdot|z)} \left[ \left\| s_t^\theta(x) - \nabla \log p_t(x|z) \right\|^2 \right] \qquad \blacktriangleright \text{ conditional score matching loss}$$

where again the difference is using the marginal score $\nabla \log p_t(x)$ vs. using the conditional score $\nabla \log p_t(x|z)$. As before, we ideally would want to minimize the score matching loss but can't because we don't know $\nabla \log p_t(x)$. But similarly as before, the denoising score matching loss is a tractable alternative:

**Theorem 22**

The score matching loss equals the denoising score matching loss up to a constant:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathcal{L}_{\text{CSM}}(\theta) + C,$$

where $C$ is independent of parameters $\theta$. Therefore, their gradients coincide:

$$\nabla_\theta \mathcal{L}_{\text{SM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CSM}}(\theta).$$

In particular, for the minimizer $\theta^*$, it will hold that $s_t^{\theta^*} = \nabla \log p_t$.

*Proof.* Note that the formula for $\nabla \log p_t$ (Equation (38)) looks the same as the formula for $u_t^{\text{target}}$ (Equation (18)). Therefore, the proof is identical to the proof of Theorem 12 replacing $u_t^{\text{target}}$ with $\nabla \log p_t$. □

**Example 23** (Denoising Diffusion Models: Score Matching for Gaussian Probability Paths)

Let us instantiate the denoising score matching loss for the case of $p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$. As we derived in Equation (40), the conditional score $\nabla \log p_t(x|z)$ has the formula

$$\nabla \log p_t(x|z) = -\frac{x - \alpha_t z}{\beta_t^2}. \tag{51}$$

Plugging in this formula, the conditional score matching loss becomes:

$$\begin{aligned}\mathcal{L}_{\text{CSM}}(\theta) &= \mathbb{E}_{t\sim\text{Unif},\, z\sim p_{\text{data}},\, x\sim p_t(\cdot|z)}\left[\left\| s_t^\theta(x) + \frac{x-\alpha_t z}{\beta_t^2}\right\|^2\right] \\ &\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif},\, z\sim p_{\text{data}},\, \epsilon\sim\mathcal{N}(0,I_d)}\left[\left\| s_t^\theta(\alpha_t z+\beta_t\epsilon) + \frac{\epsilon}{\beta_t}\right\|^2\right] \\ &= \mathbb{E}_{t\sim\text{Unif},\, z\sim p_{\text{data}},\, \epsilon\sim\mathcal{N}(0,I_d)}\left[\frac{1}{\beta_t^2}\left\|\beta_t s_t^\theta(\alpha_t z+\beta_t\epsilon) + \epsilon\right\|^2\right]\end{aligned}$$

where in $(i)$ we plugged in Equation (28) and replaced $x$ by $\alpha_t z + \beta_t\epsilon$. Note that the network $s_t^\theta$ essentially learns to predict the noise that was used to corrupt a data sample $z$. This explains why the above training loss is called **denoising score matching**. It was soon realized that the above loss is numerically unstable for $\beta_t \approx 0$ close to zero (i.e. denoising score matching only works if you add a sufficient amount of noise). In some of the first works on denoising diffusion models (see **Denoising Diffusion Probabilitic Models**, [17]) it was therefore proprosed to drop the constant $\frac{1}{\beta_t^2}$ in the loss and reparameterize $s_t^\theta$ into a **noise predictor** network $\epsilon_t^\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ via:

$$-\beta_t s_t^\theta(x) = \epsilon_t^\theta(x) \quad \Rightarrow \quad \mathcal{L}_{\text{DDPM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0,I_d)}\left[\|\epsilon_t^\theta(\alpha_t z+\beta_t\epsilon)-\epsilon\|^2\right]$$

As before, the network $\epsilon_t^\theta$ essentially learns to predict the noise that was used to corrupt a data sample $z$. In Algorithm 4, we summarize the training procedure.

**Algorithm 4** Score Matching Training Procedure for Gaussian probability path

**Require:** A dataset of samples $z \sim p_{\text{data}}$, score network $s_t^\theta$ or noise predictor $\epsilon_t^\theta$

1: **for** each mini-batch of data **do**
2: Sample a data example $z$ from the dataset.
3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
5: Set $x_t = \alpha_t z + \beta_t\epsilon$ (General case: $x_t \sim p_t(\cdot|z)$)
6: Compute loss

$$\mathcal{L}(\theta) = \|s_t^\theta(x_t) + \frac{\epsilon}{\beta_t}\|^2 \qquad \text{(General case: } = \|s_t^\theta(x_t) - \nabla \log p_t(x_t|z)\|^2\text{)}$$

$$\text{Alternatively: } \mathcal{L}(\theta) = \|\epsilon_t^\theta(x_t) - \epsilon\|^2$$

7: Update the model parameters $\theta$ via gradient descent on $\mathcal{L}(\theta)$.
8: **end for**

Let us summarize the results of this section:

**Summary 24** (Score Functions, Score Matching, and Stochastic Sampling)
Let $p_t(x|z), p_t(x)$ be the conditional and marginal probability path. The **conditional score function** is given by $\nabla \log p_t(x|z)$ and the **marginal score function** is given by $\nabla \log p_t(x)$. For every diffusion coefficient $\sigma_t \geq 0$,

the trajectories of the following SDE follow the probability path:

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = \left[ u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] \mathrm{d}t + \sigma_t \mathrm{d}W_t \tag{52}$$

$$\Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1), \tag{53}$$

where is $u_t^{\text{target}}(x)$ be the marginal vector field as before (see Equation (18)).

**Score Matching.** To learn the marginal score function $\nabla \log p_t(x)$, we can use a **score network** $s_t^\theta$ and train it via **denoising score matching**

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}},\, t \sim \text{Unif},\, x \sim p_t(\cdot|z)} [\| s_t^\theta(x) - \nabla \log p_t(x|z) \|^2] \qquad \text{(denoising score matching loss)} \tag{54}$$

**Gaussian Probability Paths.** For the - most important - case of a Gaussian probability path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$, there is no need to train $s_t^\theta$ and $u_t^\theta$ separately as we can convert them via the formula:

$$u_t^\theta(x) = a_t s_t^\theta(x) + b_t x, \quad a_t = \left( \beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right), b_t = \frac{\dot{\alpha}_t}{\alpha_t}$$

After training, we can simulate the following SDE

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = \left[ \left( 1 + \frac{\sigma_t^2}{2a_t} \right) u_t^\theta(X_t) - \frac{\sigma_t^2 b_t}{2a_t} X_t \right] \mathrm{d}t + \sigma_t \mathrm{d}W_t \tag{55}$$

$$= \left[ \left( a_t + \frac{\sigma_t^2}{2} \right) s_t^\theta(X_t) + b_t X_t \right] \mathrm{d}t + \sigma_t \mathrm{d}W_t \tag{56}$$

for any diffusion coefficient $\sigma_t \geq 0$ to obtain approximate samples $X_1 \sim p_{\text{data}}$. One can empirically find the optimal $\sigma_t \geq 0$.